Image Processing Project1

311553059 孫郁茹

1. 雜訊消除：

   原圖為一張加入 salt and pepper noise 的咖哩飯照片。底下將會使用 average filter(3 x 3, 5 x 5, 7 x 7)以及 median filter(3 x 3, 5 x 5, 7 x 7)來達到降噪的目的。

   a、 原圖：可見圖片上有許多雜訊。



b、 Average filter：效果並不是很顯著，還是可以看到雜訊點，且隨著 filter 的大小越大，圖片只有更模糊，降噪效果也沒有更好。

3x3 **5x5** **7x7**



c、 Median filter：效果較 averager filter 好很多，除了降低了 noise 的干擾，細節部分的保留也較完整。

3x3 **5x5** **7x7**



2. 邊緣銳化：

將一張雪花冰的照片做邊緣銳化，除了可以觀察雪花冰上的爆米香，也可以觀察碗上的黑白條紋。首先將圖片用 average filter 模糊化得到 blur_pic，再將原圖減去 blur_pic 即可得到 detail_pic，最後將原圖加上 detail_pic 即可得到邊緣銳化的圖片。

底下將用一張正常亮度的圖片以及亮度提高的圖片來做銳化。

a、 正常亮度原圖&亮度提高原圖

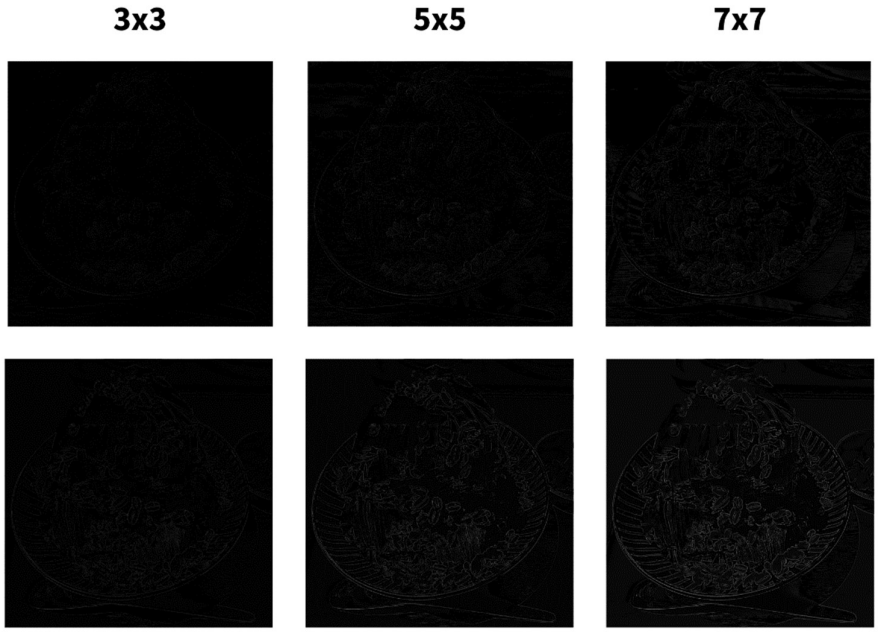
正常亮度



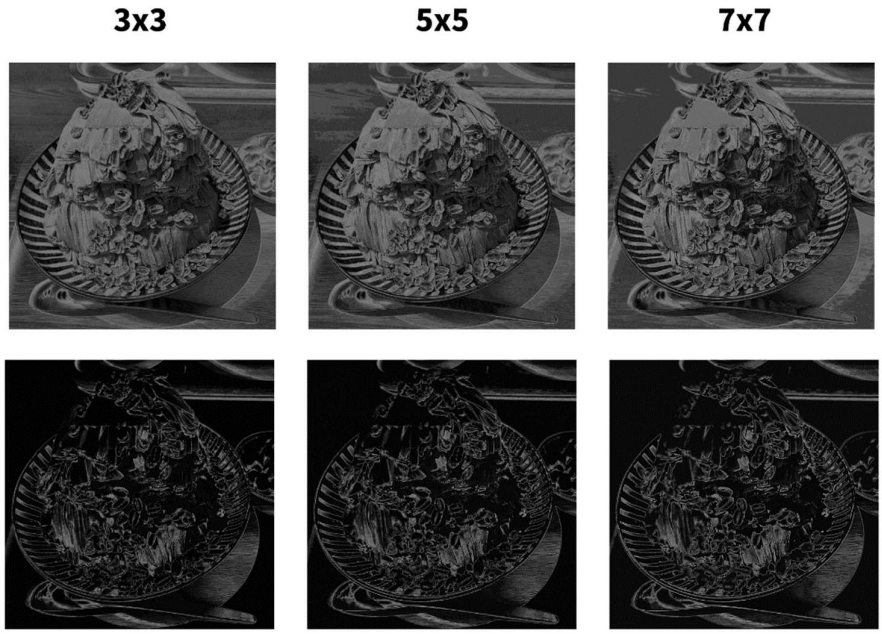
提高亮度



b、 detail（上排為正常亮度，下排為提高亮度）:

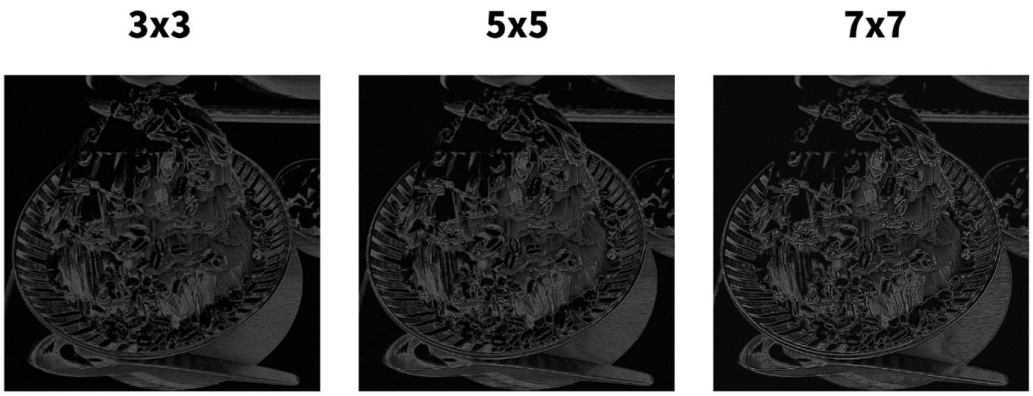


c、 原圖+detail（上排為正常亮度，下排為提高亮度）:



我們可以觀察到在 detail 部分，亮度提高的照片表現較佳，但在加入原圖後，卻丟失了部分的細節，許多部份變成漆黑一片。

d、 亮度正常原圖+提高亮度 detail：



因正常亮度的圖片在碗的邊緣的銳化表現並不好，而亮度提高的圖片丟失太多原有細節，於是我將正常亮度的原圖加上亮度提高的 detail，得到以上的圖片。

3. 亮度提高：

原圖為一張在傍晚拍攝的跑步照片，我將使用不同參數的 power law transformation 以及 histogran equalization 來提高圖片亮度並比較效果。

a、 原圖



b、 分別使用 gamma=1.2, 1.5, 2.0 來提高圖片亮度：

1.2



1.5



2.0



c、 使用 histogram equalization：



比較使用 power law transformation 的三張圖片後可以發現當 gamma=1.2 時亮度最剛好，其他圖片都過亮了。

而 histogram equalization 在此圖的效果相較於 power law 並沒有太好，圖片整體變得有一些過亮。

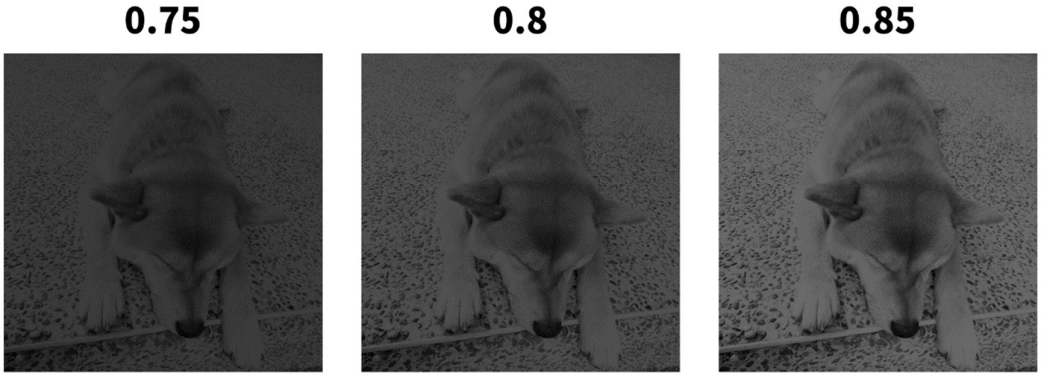
4. 亮度降低：

原圖為一張過亮的狗狗照片，我將使用不同參數的 power law transformation 以及 histogran equalization 來降地圖片亮度並比較效果。

a、 原圖



b、 分別使用 gamma=0.75, 0.8, 0.85 來提高圖片亮度：



c、 使用 histogram equalization：



比較使用 power law transformation 的三張圖片後可以發現當 gamma=0.8 時亮度最剛好，但這些圖片卻有一層灰矇矇的感覺。

而 histogram equalization 在此圖的效果反而較佳，有將狗以及石頭地板的細節表現出來，也沒有一層暗暗的感覺。

由第三點以及第四點可知，兩種調整亮度的方法並不一定誰好誰壞，取決於處理的照片都會產生不同的效果。

```
function curry()
% apply average filter & median filter (salt and pepper noise)
f = imread('curry_gray.jpg');
s = size(f);

% step0: add salt&pepper noise to image
f0 = f;
noise_percent = 10;
for i = 1:s(1)
    for j=1:s(2)
        noise_check = randi(noise_percent);
        if (noise_check == noise_percent)
            f0(i, j) = randi(256);
        end
    end
end
imwrite(f0, 'curry_salt10.jpg');

% step1: denoise with average filter
blur_3 = ones(3, 3)/9;
blur_5 = ones(5, 5)/25;
blur_7 = ones(7, 7)/49;

f1 = f0; f2 = f0; f3 = f0;

for i = 1:s(1)
    for j=1:s(2)
        f1(i,j) = my_spatial_filter(f0, i, j, blur_3, 3);
        f2(i,j) = my_spatial_filter(f0, i, j, blur_5, 5);
        f3(i,j) = my_spatial_filter(f0, i, j, blur_7, 7);
    end
end

% step2: output
imwrite(f1, 'curry_blur3.jpg');
imwrite(f2, 'curry_blur5.jpg');
imwrite(f3, 'curry_blur7.jpg');
```

```
% step3: denoise with msdian filter
f4 = f0; f5 = f0; f6 = f0;
for i = 1:s(1)
    for j=1:s(2)
        f4(i,j) = my_median_filter(f0, i, j, 3);
        f5(i,j) = my_median_filter(f0, i, j, 5);
        f6(i,j) = my_median_filter(f0, i, j, 7);
    end
end

end
```

```
function ice()
% sharpen edges of ice_gray.jpg

f = imread('ice_gray.jpg');
fb = imread('ice_gray_b.jpg');
s = size(f);

f1 = f; f2 = f; f3 = f;
f1b = f; f2b = f; f3b = f;
% step1: calculate blur matrix
blur_3 = ones(3, 3)/9;
blur_5 = ones(5, 5)/25;
blur_7 = ones(7, 7)/49;

% step2: calculate blurred image
for i=1:s(1)
    for j=1:s(2)
        f1(i,j) = my_spatial_filter(f, i, j, blur_3, 3);
        f2(i,j) = my_spatial_filter(f, i, j, blur_5, 5);
        f3(i,j) = my_spatial_filter(f, i, j, blur_7, 7);
        f1b(i,j) = my_spatial_filter(fb, i, j, blur_3, 3);
        f2b(i,j) = my_spatial_filter(fb, i, j, blur_5, 5);
        f3b(i,j) = my_spatial_filter(fb, i, j, blur_7, 7);
    end
end

% step3-1: output blurred image
%imwrite(f1, 'ice_blur3.jpg');
%imwrite(f2, 'ice_blur5.jpg');
%imwrite(f3, 'ice_blur7.jpg');

% step3-2: output detail image (original - blurred)
imwrite(f-f1, 'ice_detail3.jpg');
imwrite(f-f2, 'ice_detail5.jpg');
imwrite(f-f3, 'ice_detail7.jpg');
imwrite(fb-f1b, 'ice_detail3b.jpg');
imwrite(fb-f2b, 'ice_detail5b.jpg');
imwrite(fb-f3b, 'ice_detail7b.jpg');
```

```
% step3-3: output sharpened image (original + detail)
imwrite(2*f-f1, 'ice_sharp3.jpg');
imwrite(2*f-f2, 'ice_sharp5.jpg');
imwrite(2*f-f3, 'ice_sharp7.jpg');
imwrite(2*fb-f1b, 'ice_sharp3b.jpg');
imwrite(2*fb-f2b, 'ice_sharp5b.jpg');
imwrite(2*fb-f3b, 'ice_sharp7b.jpg');
imwrite(f+fb-f1b, 'ice_sharp3mix.jpg');
imwrite(f+fb-f2b, 'ice_sharp5mix.jpg');
imwrite(f+fb-f3b, 'ice_sharp7mix.jpg');
end
```

```
function running()
% 提高run_gary.jpg亮度

f = imread('run_gray.jpg');
s = size(f);
f2 = f; f3 = f; f4 = f; f5 = f;

% power law
for i=1:s(1)
    for j=1:s(2)
        f2(i,j) = power_law_trans(f(i, j), 0.8);
        f3(i,j) = power_law_trans(f(i, j), 1.2);
        f4(i,j) = power_law_trans(f(i, j), 1.5);
        f5(i,j) = power_law_trans(f(i, j), 2);
    end
end

imwrite(f2, 'run_adjust0.8.jpg');
imwrite(f3, 'run_adjust1.2.jpg');
imwrite(f4, 'run_adjust1.5.jpg');
imwrite(f5, 'run_adjust2.0.jpg');

% histogran equalization
f = hist_equa(f);
imwrite(f, 'run_histEqua.jpg');

end
```

```
function dog()
% 降地dog_gary.jpg亮度
f = imread('dog_gray.jpg');
s = size(f);
f2 = f; f3 = f; f4 = f; f5 = f;

% power law
for i=1:s(1)
    for j=1:s(2)
        f2(i,j) = power_law_trans(f(i, j), 0.70);
        f3(i,j) = power_law_trans(f(i, j), 0.75);
        f4(i,j) = power_law_trans(f(i, j), 0.80);
        f5(i,j) = power_law_trans(f(i, j), 0.85);
    end
end

imwrite(f2, 'dog_adjust0.70.jpg');
imwrite(f3, 'dog_adjust0.75.jpg');
imwrite(f4, 'dog_adjust0.80.jpg');
imwrite(f5, 'dog_adjust0.85.jpg');

% histogran equalization
f = hist_equa(f);
imwrite(f, 'dog_histEqua.jpg');

end
```

```
function n = my_spatial_filter(image, x, y, filter, filter_size)
% function to apply spatial filter to image
s = size(image);
image_size = s(1);
range = (filter_size-1)/2;
index=1;
n = 0;
for i = x-range:x+range
    for j = y-range:y+range
        % in origianl image range
        if (i > 0 && i <= image_size && j > 0 && j <= image_size)
            n = n + filter(index) * image(i, j);
            if (n < 0)
                n = 0;
            elseif (n > 255)
                n = 255;
            end
        end
        index = index + 1;
    end
end

end
```

```
function s = power_law_trans(r, gamma)
 % function to power law translation to image
s = double(r)^gamma;

if (s < 0)
    s = 0;
elseif (s > 256)
    s = 256;
end

end
```

```
function n = my_median_filter(image, x, y, filter_size)
% function to apply median filter to image
s = size(image);
image_size = s(1);
range = (filter_size-1)/2;

index = 1;
arr = [];
for i = x-range:x+range
    for j = y-range:y+range
        if (i > 0 && i <= image_size && j > 0 && j <= image_size)
            arr(index) = image(i, j);
            index = index + 1;
        end
    end
end

n = median(arr, 'all');

end
```

```
function n = hist_equa(f)
% function to apply histogram equalization to image
s = size(f);
h = zeros(1,256);
for i=1:s(1)
    for j=1:s(2)
        h(f(i,j) + 1) = h(f(i,j) + 1) + 1;
    end
end

for i=1:256
    h(i) = h(i)/(s(1)*s(2));
end



for i=2:256
    h(i) = h(i-1) + h(i);
end

n=f;
for i=1:s(1)
    for j=1:s(2)
        n(i,j) = round(h(f(i,j)+1)*255);
    end
end
```